

リスト、タプル

リスト

リストとは？～複数の値をまとめる箱～

変数に1つずつ入れると大変な時、リストを使うとスッキリ管理できます

リストとは？

複数の値をまとめて
1つの変数に入れられる箱

例：クラスの名前リスト
["田中","佐藤","山田"]

変数に1つずつ入れると
管理が大変になる...

日常例で考えると

 買い物リスト

- ・りんご
- ・バナナ
- ・みかん

→ これをPythonで表すと
["りんご","バナナ","みかん"]



複数のデータをまとめて管理したいとき → リストの出番！

リストの作り方と要素の型

[] で作成し、カンマで区切ります。異なる型の値を混ぜることもできます

```
# リストの作り方：[ ] で囲み、カンマで区切る
```

```
fruits = ["りんご", "バナナ", "みかん"]
```

```
scores = [80, 95, 72, 88]
```

```
# 異なる型を混ぜることもできる
```

```
mixed = ["山田", 25, True]    # str・int・boolが混在
```

```
# 空のリストも作れる
```

```
empty = []
```

ポイント

[] の中にカンマで区切って値を並べる
文字列・数値・真偽値など
異なる型を混ぜてもOK

変数との違い

変数：1つの値を入れる箱

リスト：複数の値を入れる箱

→ まとめて管理できる！

インデックスで要素を取り出す

各要素には0から始まる番号(インデックス)が振られています

```
fruits = ["りんご", "バナナ", "みかん"]  
#       0      1      2      ← インデックス番号
```

```
print(fruits[0])    # → りんご (最初は0番！)  
print(fruits[1])    # → バナナ  
print(fruits[2])    # → みかん
```

⚠ 0始まりに注意！

最初の要素は [1] ではなく [0]

人間の数え方 → 1番目

Pythonの数え方 → 0番目

最初だけ覚えれば
あとは順番通り！

存在しない番号はエラー

```
fruits = ["りんご", "バナナ", "みかん"]  
# 要素は3つ (0・1・2番)
```

```
fruits[3] → ❌ エラー！  
IndexError が発生
```

→ len()で確認してから
アクセスするのが安全

リストは変更できる(書き換え・追加)

インデックスを使って書き換え、appendで末尾に追加できます

```
fruits = ["りんご", "バナナ", "みかん"]

# 要素の書き換え (インデックスを指定して上書き)
fruits[1] = "ぶどう"
print(fruits)    # → ["りんご", "ぶどう", "みかん"]

# 要素の追加 (末尾に追加)
fruits.append("もも")
print(fruits)    # → ["りんご", "ぶどう", "みかん", "もも"]
```

書き換え

リスト[番号] = 新しい値
指定した番号の要素を上書きする

追加 (append)

リスト.append(追加する値)
リストの末尾に新しい要素を追加する

リストは変更できる(削除・取り出し)

removeで削除、popで要素を取り出しながら削除ができます。

```
fruits = ["りんご", "ぶどう", "みかん", "もも"]
```

```
# 要素の削除
```

```
fruits.remove("ぶどう")
```

```
print(fruits)    # → ["りんご", "みかん", "もも"]
```

```
# 要素の取り出し (インデックスから要素を取り出し、削除)
```

```
item=fruits.pop()
```

```
print(item)      # → もも
```

```
print(fruits)    # → ["りんご", "みかん"]
```

削除 (remove)

リスト.remove(削除する値)

リストから指定し値を削除する

削除しながら取得(pop)

リスト.pop()

リストの末尾()の値を取得し、

リストからは削除

※pop(1)のように番号を指定することも可能

リストの長さを調べる

len() でリストの要素数を取得できます。文字列と同じ使い方です

```
fruits = ["りんご", "バナナ", "みかん"]  
print(len(fruits))    # → 3
```

```
scores = [80, 95, 72, 88, 64]  
print(len(scores))    # → 5
```

len() の使いどころ

リストの要素数を確認したい
ループ処理の回数を決めるとき
空かどうかのチェック

```
len(リスト) == 0  
→ 空のリスト
```

文字列でも同じ！

len() は文字列にも使える

```
len("Hello") → 5  
len(fruits) → 3
```

→ 覚え方は同じ！



for文と組み合わせると、リストの全要素を順番に処理できます
(2日目PMで学習！)

やってみよう:リスト

以下のコードを書いて実行してみましょう

① 自分の好きな食べ物を3つリストに入れよう

```
foods = ["〇〇", "〇〇", "〇〇"]
```

```
print(foods)
```

```
print(len(foods))    # 要素数を確認
```

② インデックスで1つ取り出してみよう

```
print(foods[0])      # 最初の食べ物
```

③ 2番目の食べ物を別の食べ物に書き換えよう

```
foods[1] = "〇〇"
```

```
print(foods)
```

④ 新しい食べ物を末尾に追加しよう

```
foods.append("〇〇")
```

```
print(foods)
```

タプル

タプルとは？作り方

() で複数の値をまとめます。リストとよく似ていますが、大きな違いがあります

```
# タプルの作り方：( ) で囲み、カンマで区切る
point = (35.6, 139.7)      # 緯度・経度",
rgb = (255, 128, 0)        # 色情報
person = ("山田", 25, "東京") # 名前・年齢・都市
```

リストとの違い

リスト [] で作る
タプル () で作る

見た目はほぼ同じ！
でも大きな違いが...
→ 次のスライドへ

タプルの特徴

インデックスで取り出せる
point = (35.6, 139.7)
point[0] → 35.6

len() も使える
len(point) → 2

タプルは変更できない(イミュータブル)

リストとの最大の違いは「後から変更できるかどうか」です

✓ リスト (変更できる)

```
fruits = ["りんご","バナナ"]  
fruits[0] = "ぶどう" → OK  
fruits.append("もも") → OK
```

後から自由に変更できる
= ミュータブル (可変)

✗ タプル (変更できない)

```
point = (35.6, 139.7)  
point[0] = 0.0  
→ TypeError ! 変更不可
```

一度作ったら変えられない
= イミュータブル (不変)

💡 タプルが「変更できない」のは欠点ではなく特徴！
「絶対に変えてはいけないデータ」を守るために使います

リストとタプルの使い分け

「変更されるか・されないか」で選びましょう

リストを使う場面

 変更される可能性がある

- ・ 買い物リスト（追加・削除あり）
- ・ ユーザーの入力データ
- ・ 処理中に増減するデータ

例：["田中","佐藤","山田"]
メンバーが変わる可能性あり

タプルを使う場面

 変更されては困るデータ

- ・ 座標・色情報（RGB）
- ・ 設定値・定数
- ・ 絶対に変えたくないデータ

例：(35.6, 139.7)
東京の緯度経度は変わらない

 迷ったらリスト。変えてはいけないと確信できるデータだけタプルを使う

アンパック(複数変数への一括代入)

タプルやリストの要素を複数の変数に一気に代入できます

```
# タプルのアンパック：複数の変数に一気に代入
point = (35.6, 139.7)
lat, lng = point
print(lat)    # → 35.6
print(lng)    # → 139.7
```

アンパックの便利さ

1行で複数の変数に代入できる

従来の書き方

```
lat = point[0]
```

```
lng = point[1]
```

アンパックを使うと

```
lat, lng = point ← スッキリ！
```

注意点

変数の数と要素の数が
一致しないとエラー！

```
a, b = (1, 2, 3)
```

→ ValueError !

要素数を確認してから
使うのが安全

やってみよう

タプルとリストを組み合わせて使ってみましょう

```
# ① タプルを作って、アンパックしてみよう
person = ("山田", 25, "東京")
name, age, city = person
print(f"{name}さんは{city}在住で{age}歳です")
```

```
# ② リストに変換してから要素を変更してみよう
person_list = list(person)
person_list[1] = 26    # 年齢を更新
print(person_list)
```

まとめ

リスト[] 変更できる・順番あり・追加削除OK → 変化するデータに使う

タプル() 変更できない・順番あり・アンパック可能 → 固定データに使う

ポイント インデックス（要素番号）は0から始まる